

Snyk Integration with Claude Code Using MCP

1. Purpose

This document explains how **Snyk integrates with Claude Code using the Model Context Protocol (MCP)**.

The objective is to understand the architecture, configuration, authentication, MCP tools, scan execution flow, result handling, security controls, and the additional Claude Code mechanisms Snyk uses to provide its **Secure at Inception** experience.

This analysis can serve as a reference for understanding how a security product such as AppScan could potentially provide similar capabilities to Claude Code.

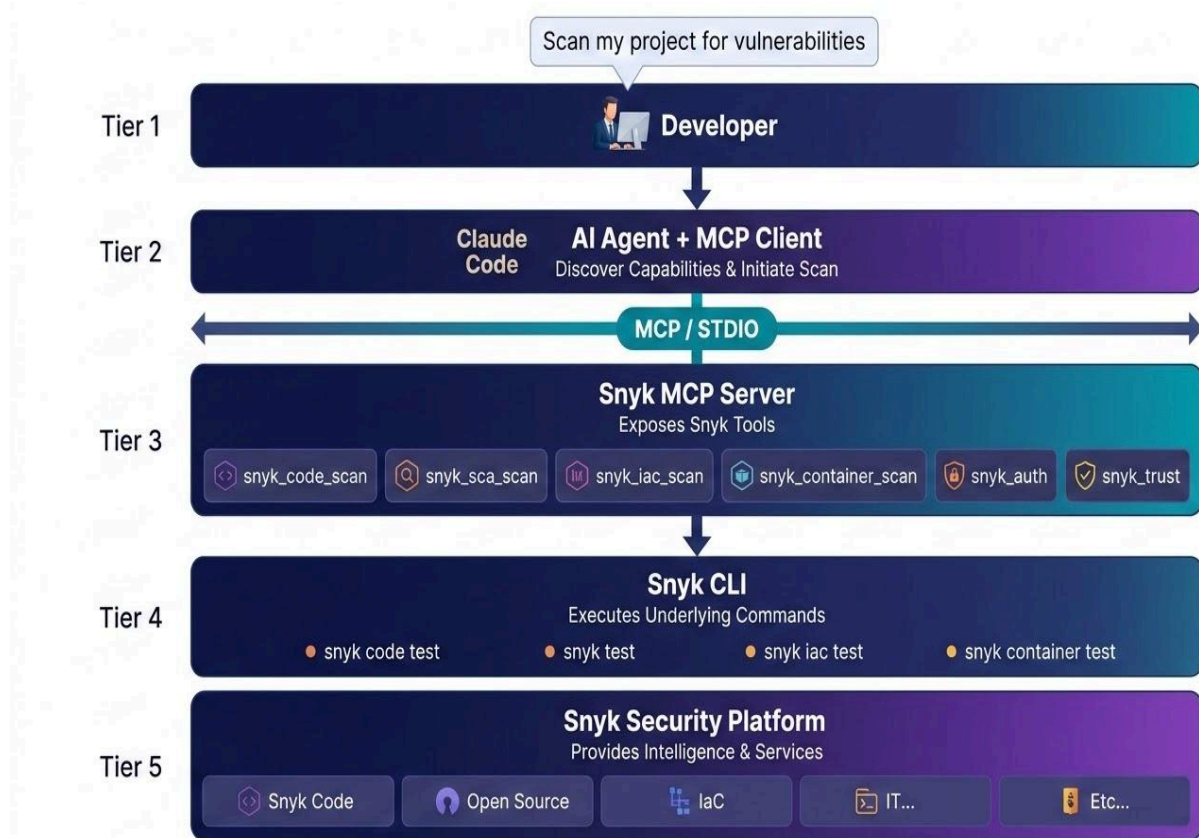
2. Executive Summary

Snyk does **not** implement a special security API inside Claude.

Instead, Snyk provides an **MCP server** that exposes Snyk security capabilities as standardized MCP tools.

Claude Code acts as the **MCP client**.

The high-level architecture is:



Snyk's documentation specifically describes its MCP server as a **local MCP server running through the Snyk CLI**. Snyk currently does not offer a hosted/remote version of this MCP server.

3. What is MCP?

MCP stands for **Model Context Protocol**.

It is an open standard that allows an AI application to communicate with external tools and services.

Instead of Claude having to understand the internal implementation of Snyk, Snyk exposes a set of standardized tools.

For example:

snyk_code_scan

means, conceptually:

"I provide a capability that can perform SAST on source code."

Claude can discover this capability and decide when to use it.

Therefore:

MCP = Standard communication mechanism between an AI agent and external capabilities

MCP itself does **not** perform security scanning.

Snyk implements the security functionality behind the MCP tools.

4. MCP Client vs MCP Server

This distinction is extremely important.

4.1 Claude Code

Claude Code is the **MCP client**.

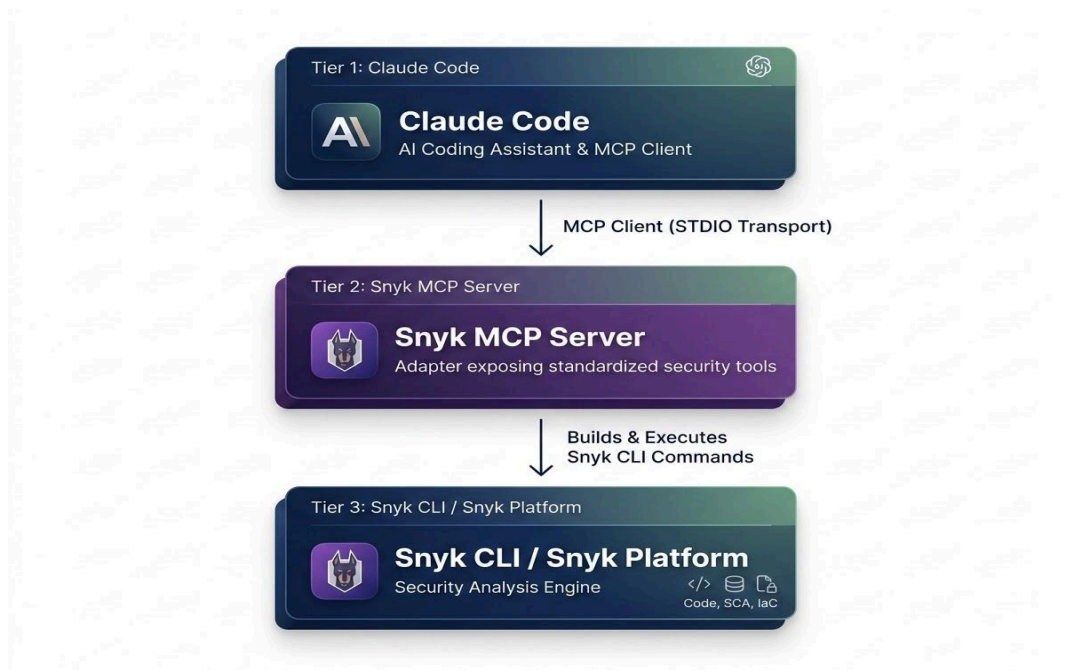
It is the AI coding agent that wants to use external capabilities.

4.2 Snyk

Snyk provides the **MCP server**.

The MCP server exposes Snyk functionality to Claude.

Therefore:



Claude does not directly understand how Snyp performs SAST.

It only needs to understand the tools exposed by the Snyp MCP server.

5. How Snyp is Registered with Claude Code

Snyp's Claude Code plugin contains an MCP configuration file:

```
{
  "mcpServers": {
    "Snyp": {
      "command": "npx",
      "args": ["-y", "snyp@latest", "mcp", "-t", "stdio"]
    }
  }
}
```

This configuration is present in Snyp's public Claude plugin repository.

The important part is:

command = npx

and:

args = -y snyk@latest mcp -t stdio

This tells Claude Code:

"When you need the Snyk MCP server, start the Snyk CLI's MCP mode using STDIO."

6. What Does the Configuration Mean?

The command is:

npx -y snyk@latest mcp -t stdio

Let's break it down.

npx

Runs an npm package.

snyk@latest

Uses the Snyk CLI package.

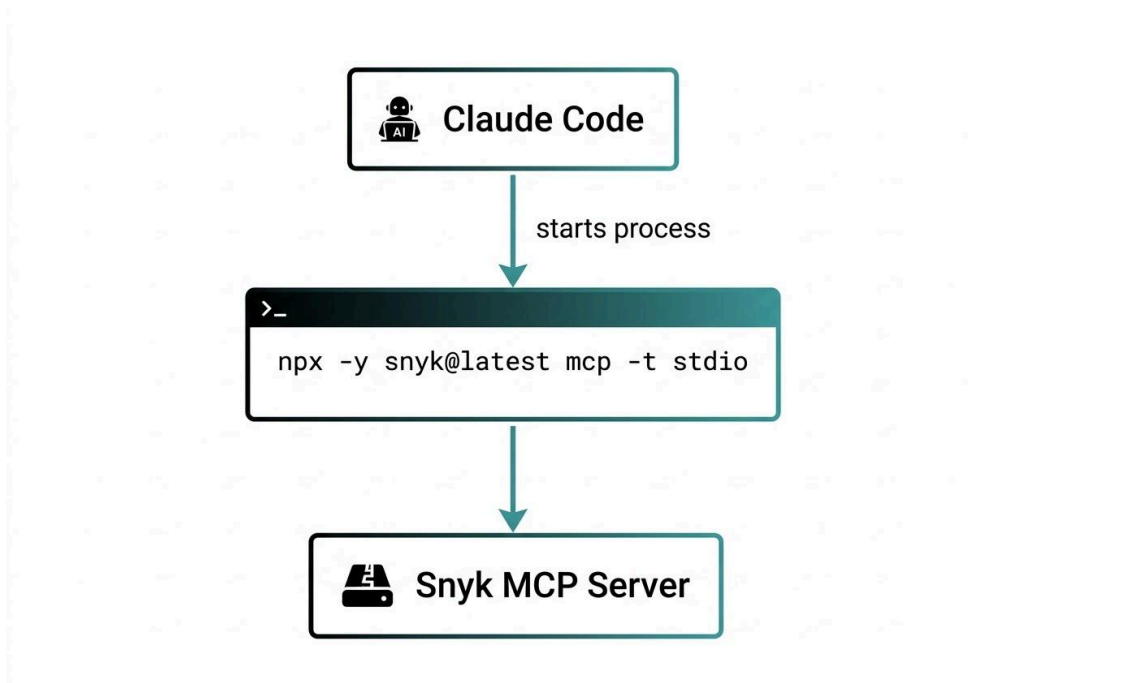
mcp

Tells the Snyk CLI to start its MCP functionality rather than execute a normal scan command directly.

-t stdio

Specifies the MCP transport as **STDIO**.

Therefore:



Snyk's current implementation accepts STDIO as its MCP transport and rejects the older retired SSE transport.

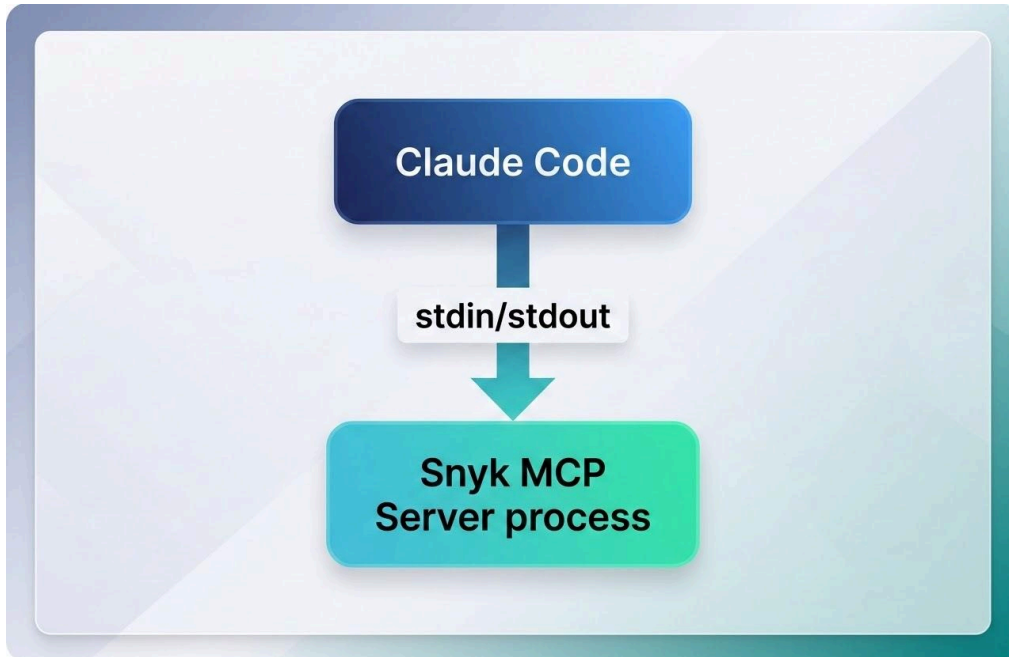
7. What is STDIO?

STDIO means:

Standard Input / Standard Output

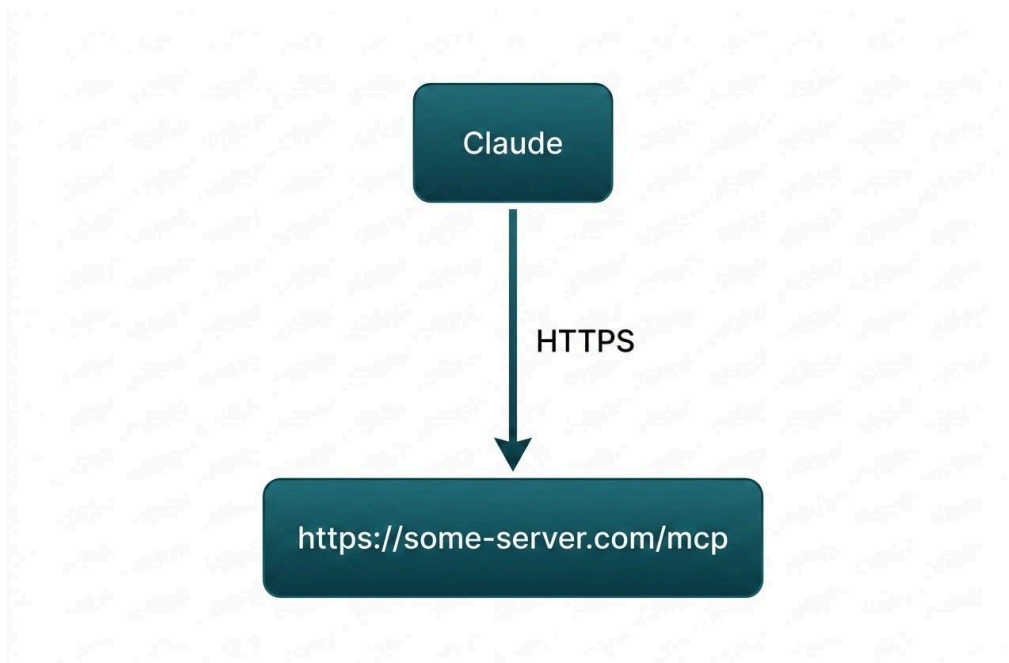
Claude Code launches the Snyk MCP server as a local process.

Conceptually:



The two processes communicate using the MCP protocol over those standard streams.

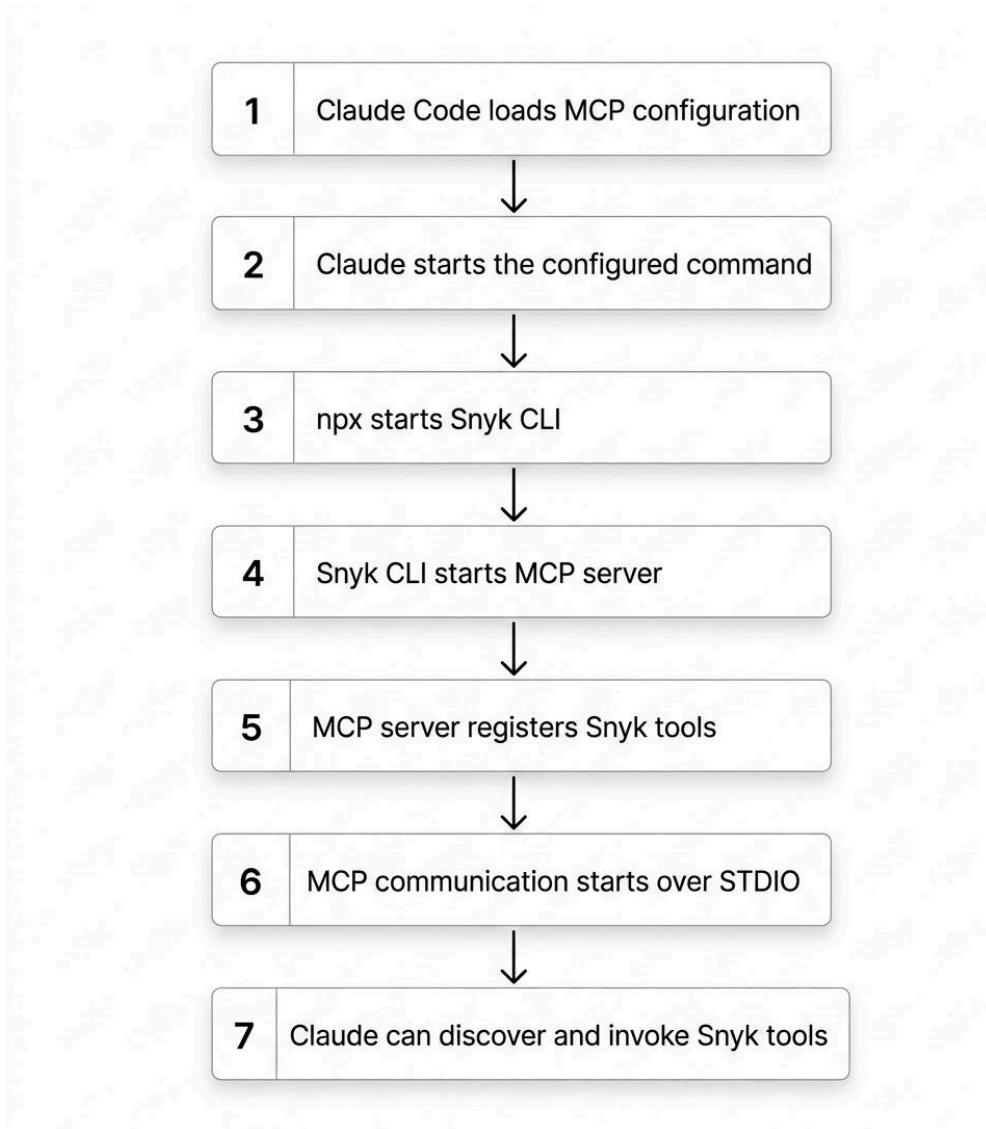
This is different from a remote architecture such as:



Snyk's current MCP architecture is **local**, specifically because it needs access to the developer's local files/projects.

8. Startup Sequence

When Claude Code starts the Snyc MCP integration, the sequence is approximately:



Snyc's implementation creates an MCP server named:

Snyc MCP Server

and registers its available tools before starting the STDIO server.

9. What Tools Does Snyk Expose?

The MCP server exposes Snyk capabilities as individual tools.

Examples include:

snyk_auth

snyk_logout

snyk_version

snyk_trust

snyk_code_scan

snyk_sca_scan

snyk_iac_scan

snyk_container_scan

snyk_sbom_scan

snyk_send_feedback

Snyk's current integration manifest documents these tools and their purposes.

The exact set can depend on the Snyk MCP profile/version.

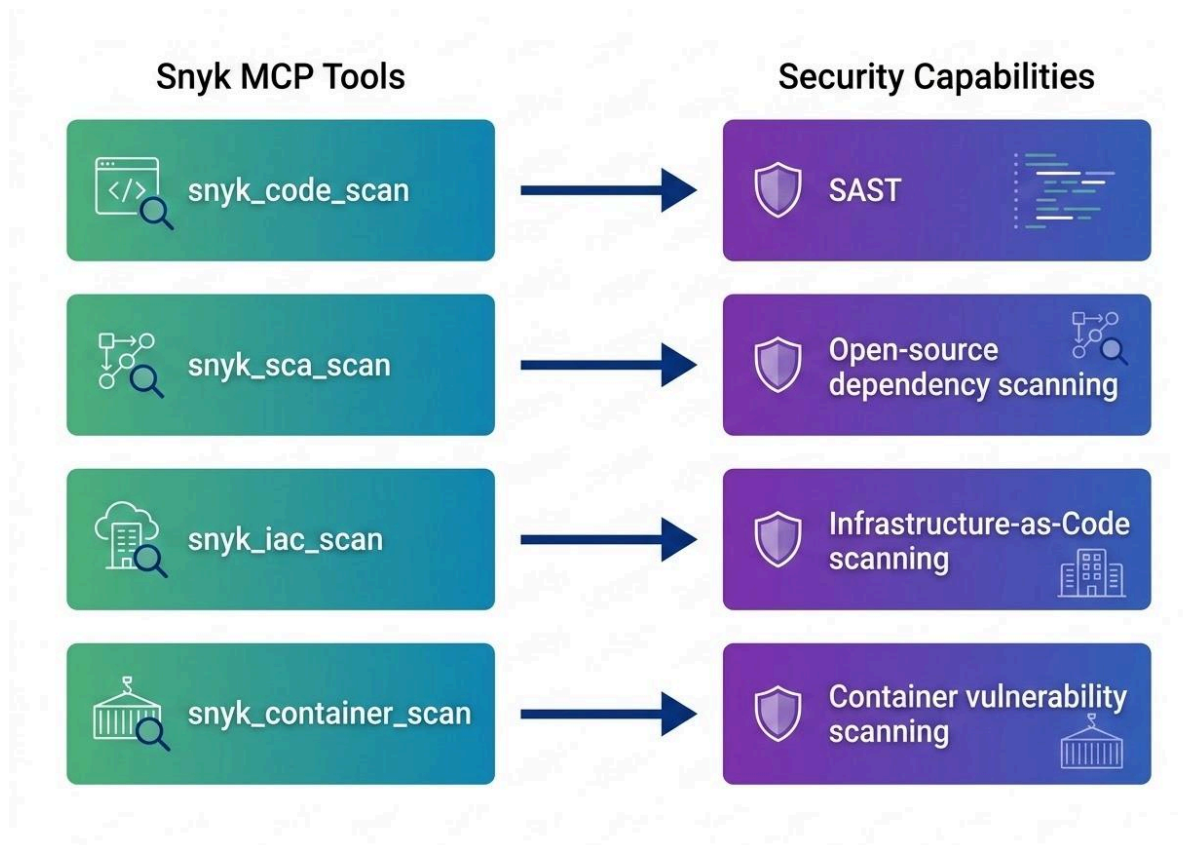
10. Why Expose Multiple Tools?

Snyk does not simply expose one generic tool such as:

snyk_scan

Instead it provides specialized capabilities.

For example:



This gives Claude semantically meaningful capabilities.

For example:

User:

"I changed some Java code."

Claude:

`snyk_code_scan` is appropriate.

User:

"I added a Maven dependency."

Claude:

`snyk_sca_scan` is appropriate.

User:

"I changed Terraform."

Claude:
snyk_iac_scan is appropriate.

11. Example: snyk_code_scan

Snyk's actual tool definition contains:

Tool name:
snyk_code_scan

It is described as performing SAST against application source code.

The tool definition maps it to the underlying Snyk CLI command:

code
test

which corresponds to:

snyk code test

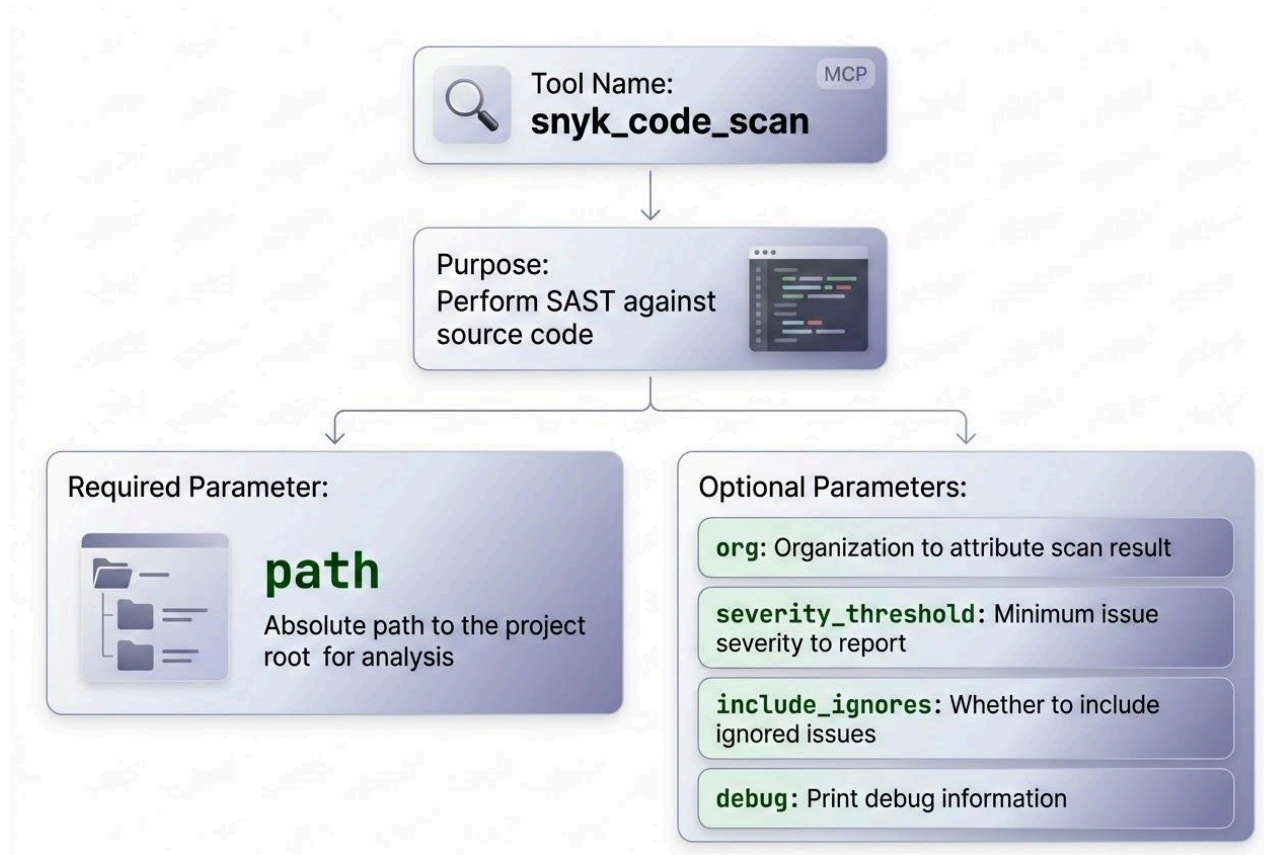
Snyk's current source explicitly defines snyk_code_scan with the command:

["code", "test"]

and defines parameters such as path, org, severity_threshold, and include_ignores.

12. Example Tool Definition

Conceptually, the tool exposed to Claude looks like:



One important detail is that Snyk explicitly instructs the AI that the `path` must be an **absolute path**.

This is a useful MCP design principle:

Tool descriptions should contain enough information for the AI to invoke the tool correctly.

13. Example: User Asks Claude to Scan Code

Suppose the developer has:

```
my-project/  
  src/  
    LoginController.java
```

UserService.java
pom.xml

The developer asks:

"Scan this project for security vulnerabilities."

Claude has access to:

snyc_code_scan
snyc_sca_scan

Claude can decide that both are relevant:

First-party code

↓

snyc_code_scan

Open-source dependencies

↓

snyc_sca_scan

For SAST, Claude could invoke something conceptually equivalent to:

```
{  
  "tool": "snyc_code_scan",  
  "arguments": {  
    "path": "/Users/developer/my-project"  
  }  
}
```

14. What Happens Inside the Snyk MCP Server?

The important point is:

Claude does not directly execute `snyc code test`.

Claude invokes:

snyk_code_scan

The Snyk MCP server receives that request.

Snyk's tool definition knows that:

snyk_code_scan

↓

snyk code test

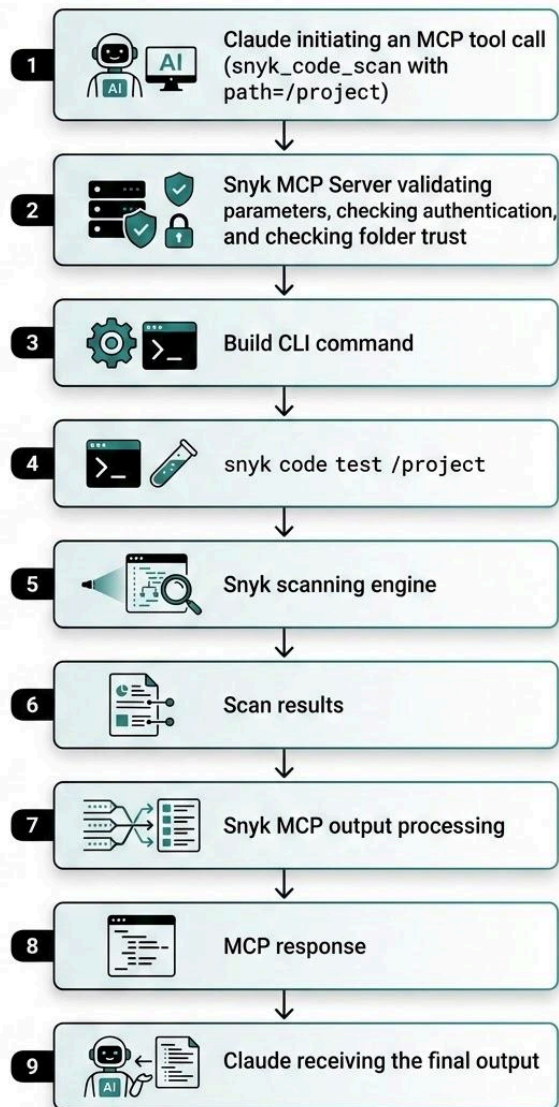
The MCP server builds the CLI command and executes it.

Snyk's source code shows that the server:

1. Receives the MCP tool request.
2. Processes the supplied parameters.
3. Performs trust/authentication checks.
4. Builds the Snyk CLI command.
5. Executes the command.
6. Captures stdout/stderr.
7. Processes the result.
8. Returns the result through MCP.

15. Detailed Execution Flow

The complete flow is:



This is the central architecture of the integration.

16. Authentication

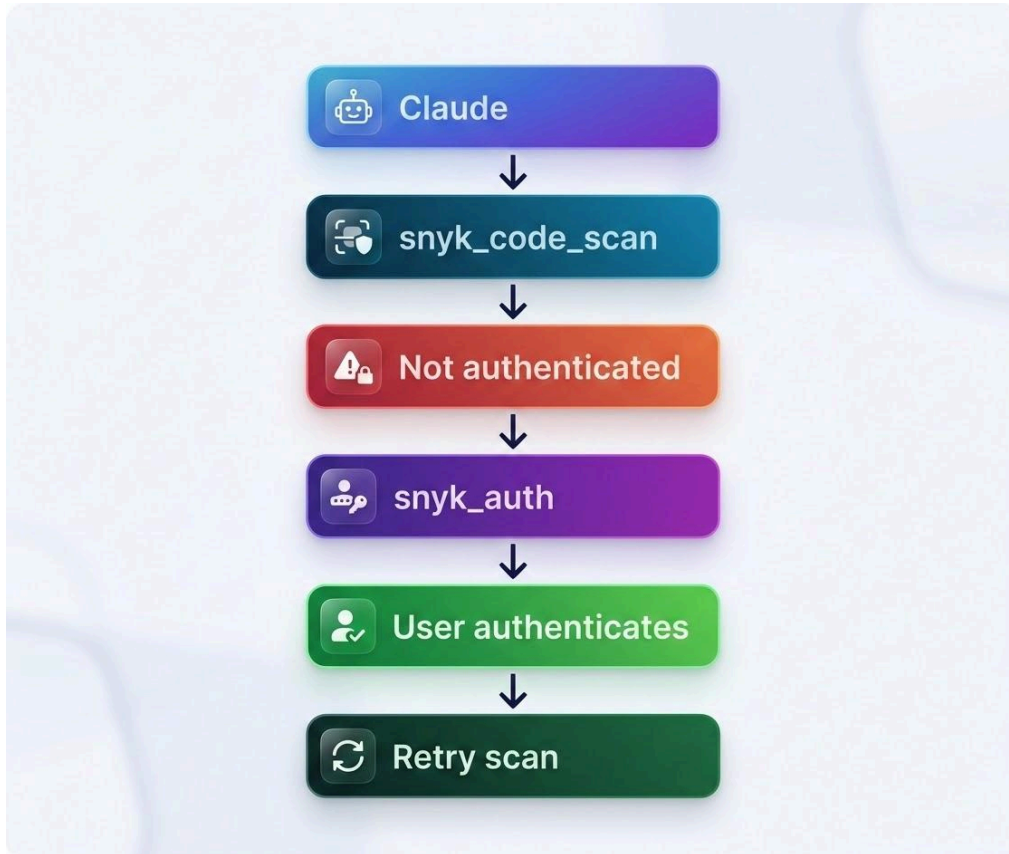
Snyk provides an MCP tool called:

snyk_auth

Its purpose is to authenticate the user with Snyk.

The current tool definition says it should be used when another Snyk tool reports that the user is not authenticated or authentication is required.

The general workflow is:



Snyk also supports supplying an API token as configuration. Its current MCP bundle documentation describes the token as an optional fallback, while interactive authentication through `snyk_auth` is recommended.

17. Folder Trust

Snyk also has another security mechanism:

`snyk_trust`

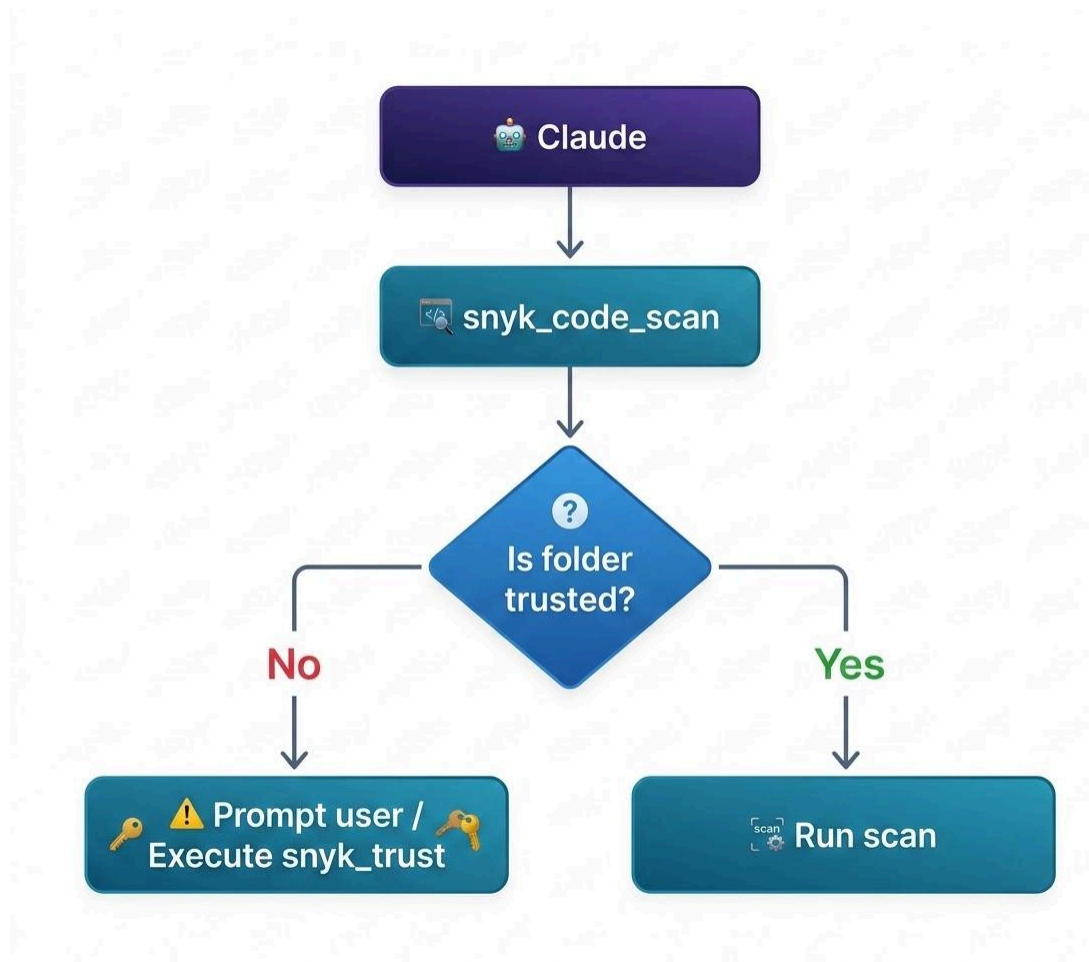
The purpose is to control which local folders Snyk is allowed to scan.

The Snyc MCP implementation checks whether the relevant folder is trusted before executing a scan. If it isn't trusted, it returns an instruction to run:

snyk_trust

first.

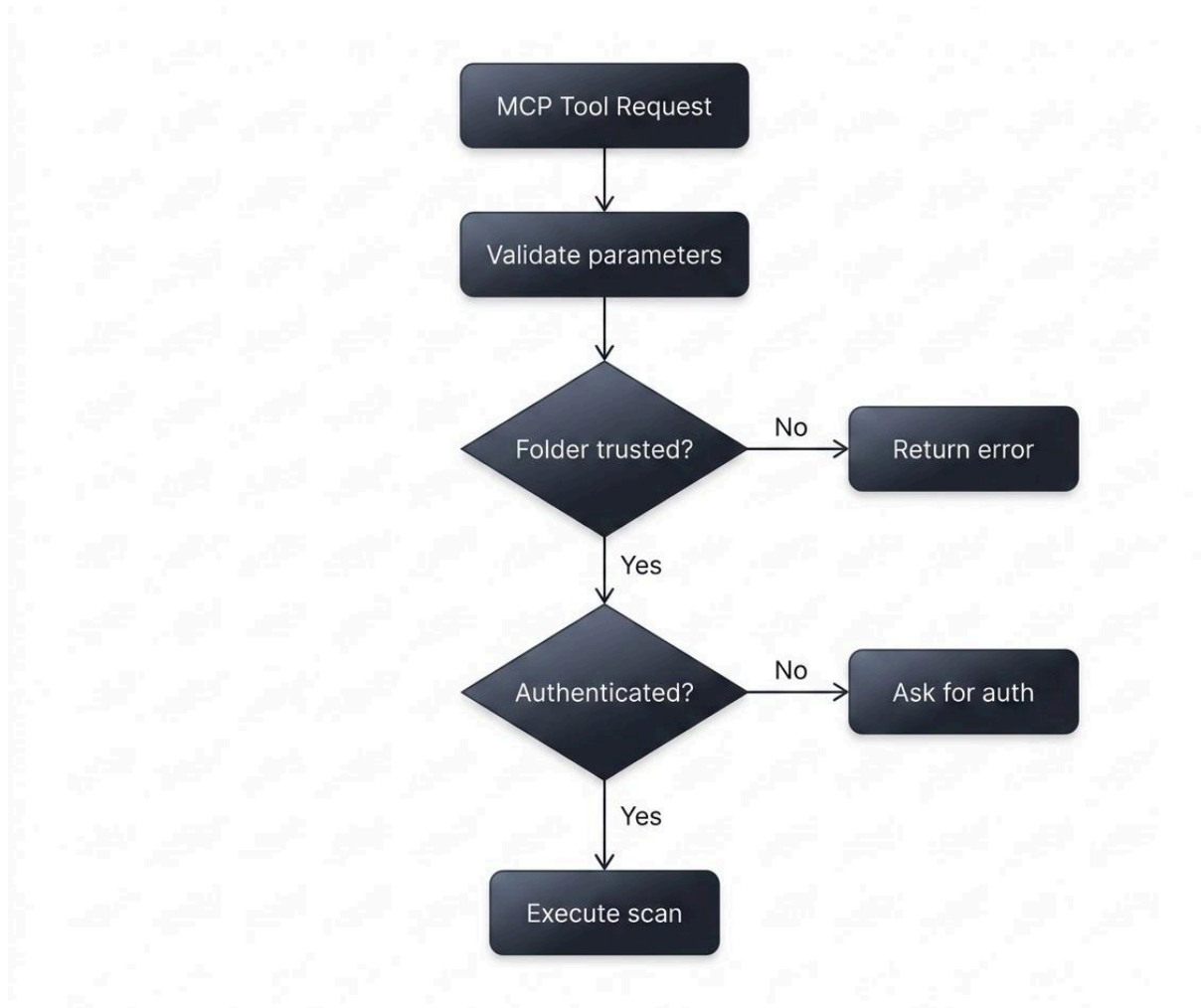
Therefore the flow can be:



This is particularly important because an AI agent is operating against files on the user's machine.

18. Authentication and Trust Checks

Before a normal Snyc scan executes, the MCP server performs checks such as:

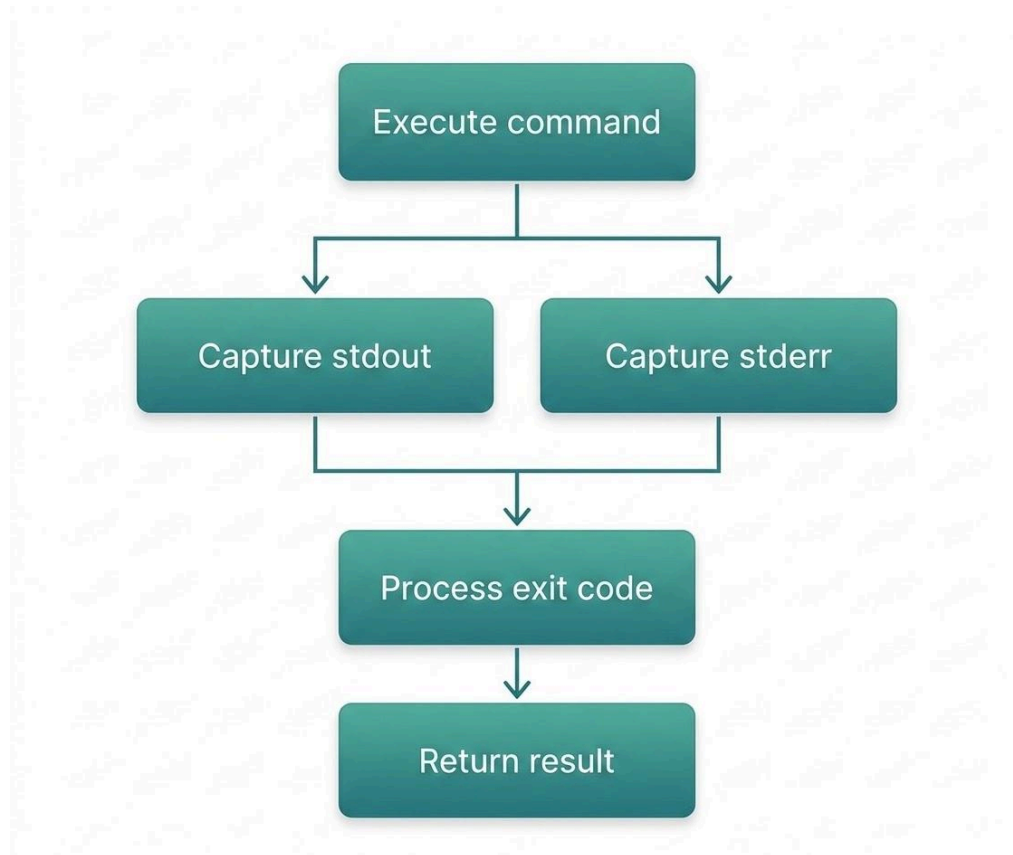


Snyk's current implementation performs both folder-trust and authentication checks in its common tool handler.

19. Execution of the Snyk CLI

The MCP server ultimately invokes the Snyk CLI as a child process.

Snyk's source uses a process execution mechanism equivalent to:



The implementation specifically captures both stdout and stderr because some Snyk CLI errors can be emitted through stderr.

Therefore, the MCP server is effectively a controlled bridge:

MCP Tool



Snyk CLI command



CLI result



MCP response

20. Example of the CLI Mapping

For SAST:

MCP:

snyk_code_scan

|

▼

Snyk command definition:

["code", "test"]

|

▼

CLI:

snyk code test <path>

For SCA:

MCP:

snyk_sca_scan

|

▼

Snyk command definition:

["test"]

|

▼

CLI:

snyk test <path>

The current Snyk tool definition explicitly contains these command mappings.

21. Why the MCP Server Does Not Simply Return Raw CLI Output

This is another important part of Snyk's implementation.

The Snyk MCP server processes/enhances scan output before returning it.

The source contains an output enhancement stage:

CLI output

↓

enhanceOutput()

↓

mapScanResponse()

↓

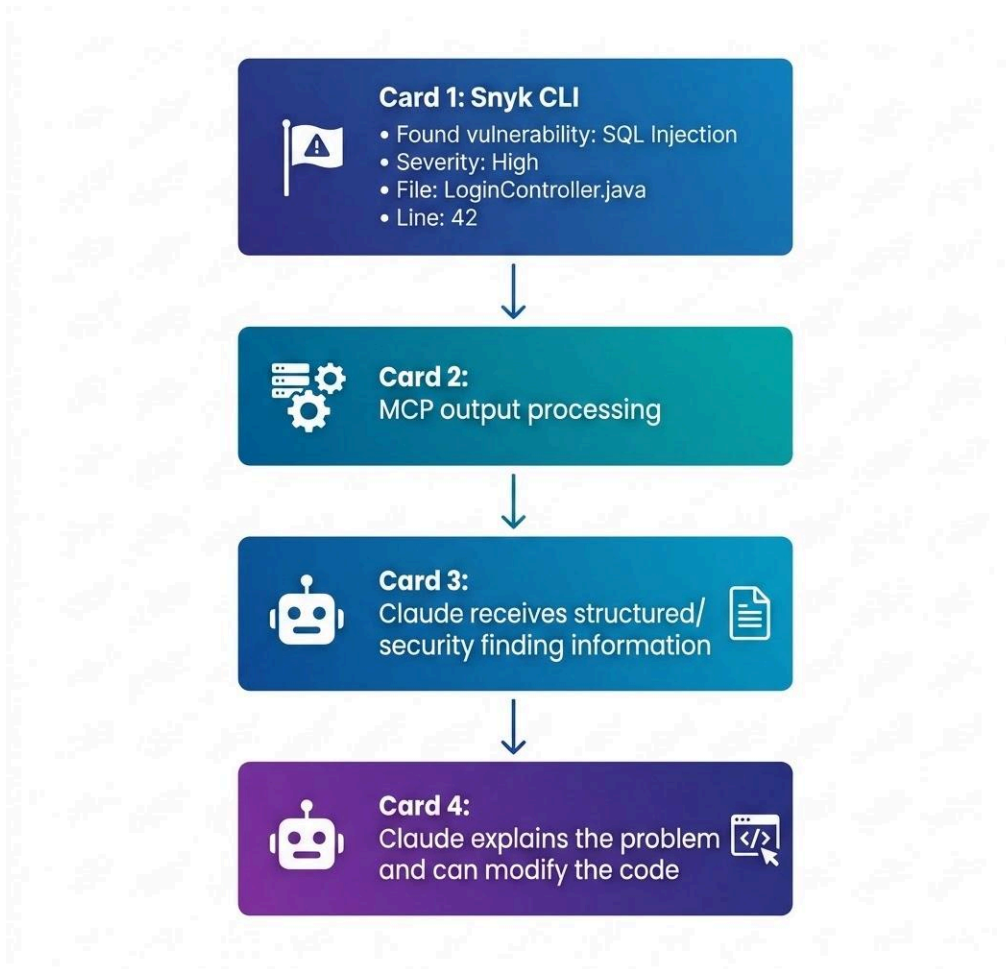
MCP result

Snyk's implementation explicitly calls an output mapping function after successful execution.

The objective is to make scan information more useful to the AI rather than simply dumping an enormous raw CLI response into Claude's context.

22. Example of the Result Flow

Conceptually:



The exact returned structure depends on the tool and current Snyk implementation, but the architectural point is important:

The MCP server acts as the adapter between Snyk's scanner output and the AI agent.

23. Example: SQL Injection Scenario

Suppose Claude generates:

```
@GetMapping("/users")
public User findUser(String username) {

    return jdbcTemplate.queryForObject(
```

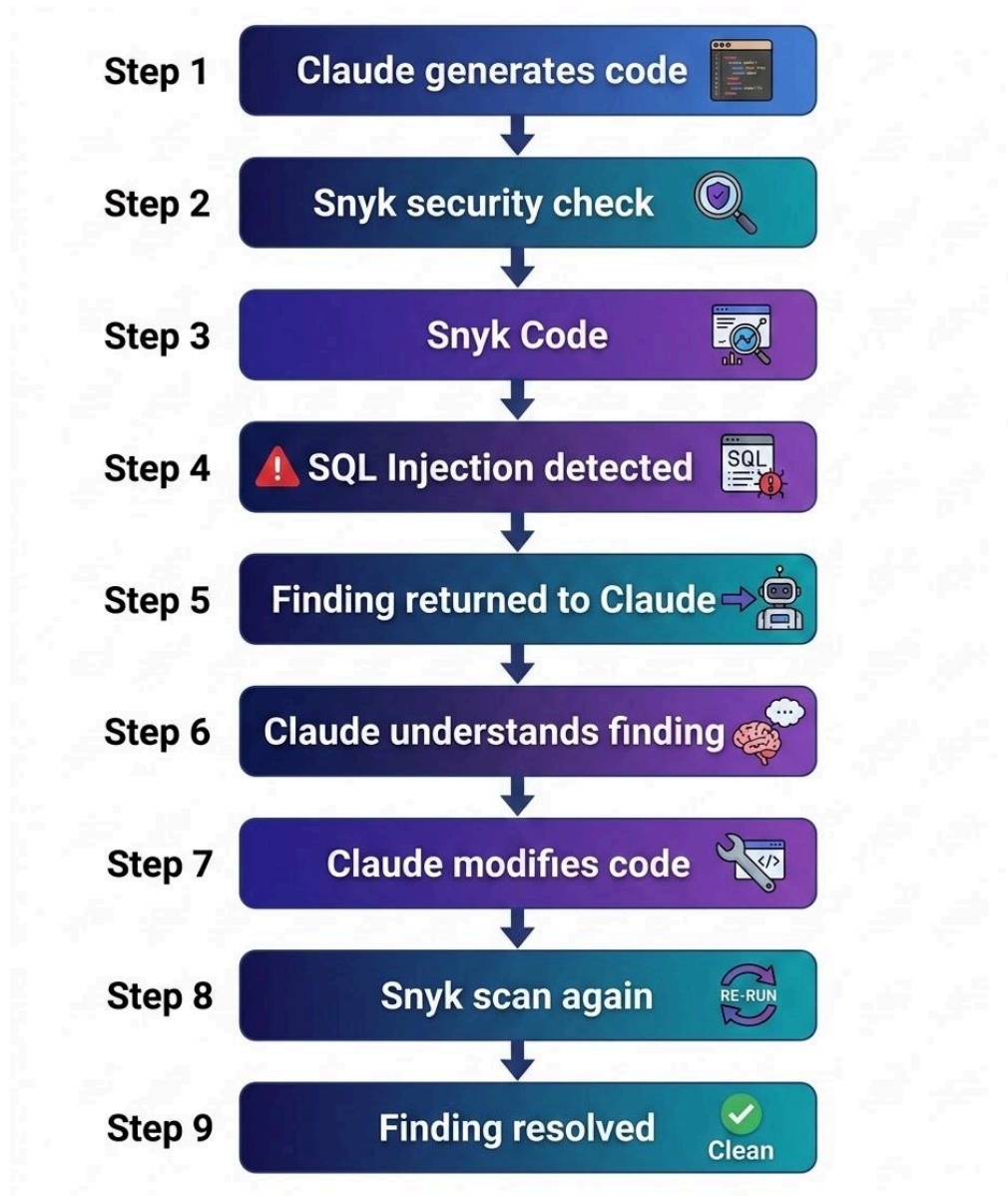
```

        "SELECT * FROM users WHERE name = '" + username + "'",
        ...
    );
}

```

This is vulnerable to SQL injection.

The Secure-at-Inception workflow can result in:



The important architectural separation is:

Snyk = Security analysis

Claude = Reasoning + coding + remediation

24. Snyk Does Not Need to Build Its Own Coding Agent

This is one of the major benefits of MCP.

Snyk does not need to build:

Snyk AI coding assistant

Instead:

Claude + Snyk MCP

creates the combined experience.

Claude already knows how to:

- understand natural language;
- inspect code;
- modify files;
- reason about changes;
- run tests;
- decide what to do next.

Snyk contributes:

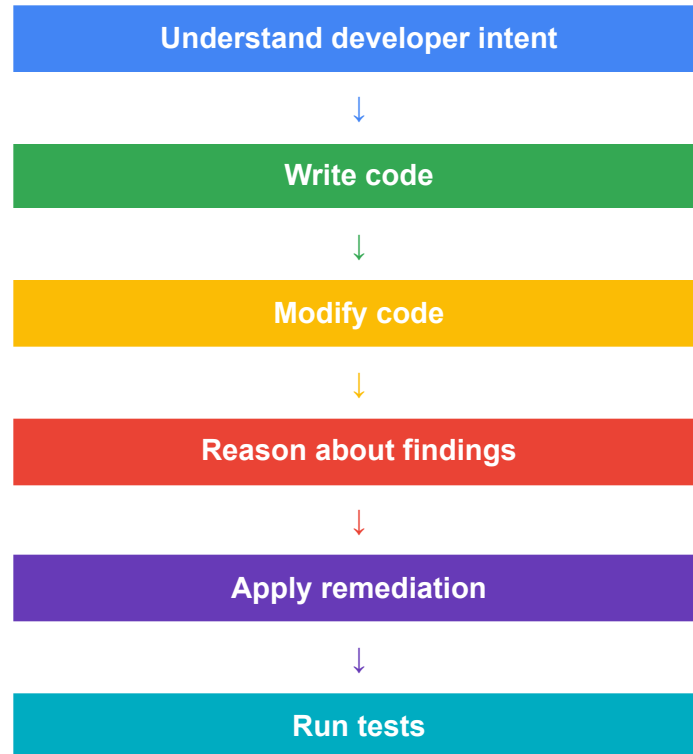
- SAST;
- SCA;
- IaC security;
- container scanning;
- vulnerability intelligence;
- security findings.

MCP connects the two.

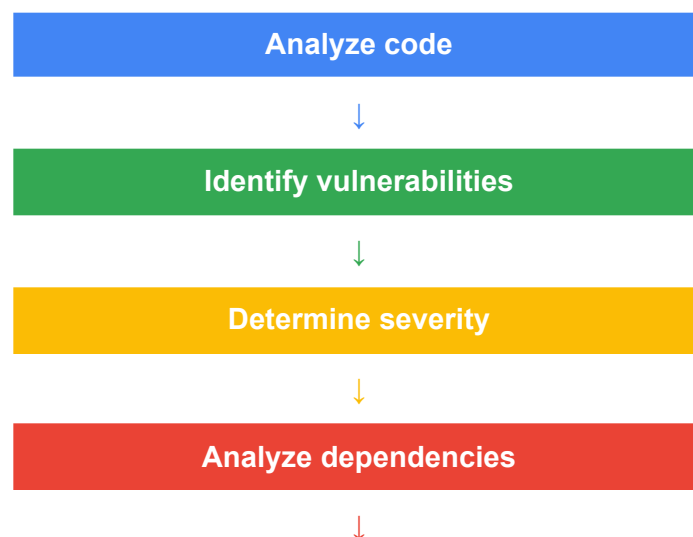
25. The Two Different Responsibilities

The architecture can therefore be viewed as:

Claude's responsibility

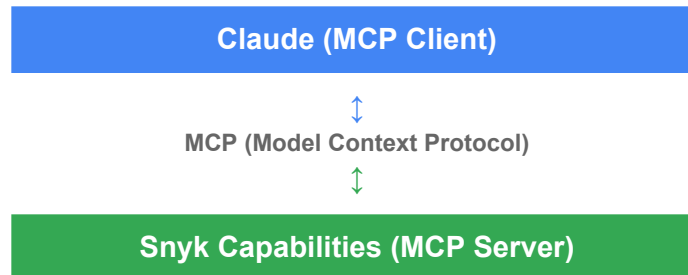


Snyk's responsibility



Provide security intelligence

MCP's responsibility



26. MCP Provides Capability; Claude Determines Usage

An important distinction is:

MCP tells Claude:

"These capabilities are available."

For example:

`snyk_code_scan`
`snyk_sca_scan`
`snyk_iac_scan`

But MCP alone doesn't necessarily mean:

"Run this tool after every code modification."

That requires an additional mechanism.

This is where Snyk's **Secure-at-Inception** implementation becomes important.

27. Secure at Inception

Snyk's current Claude Code plugin goes beyond simply making the MCP tools available.

The plugin contains:

Snyk MCP server
+
Claude Code hooks
+
/snyk-fix command
+
Secure Dependency Health Check

Snyk describes the plugin as integrating security scanning directly into the agentic development loop.

The repository specifically states that Secure-at-Inception hooks run background Snyk scans on file edits and can prevent Claude from stopping when newly introduced vulnerabilities are found.

28. MCP vs Claude Code Hooks

This distinction is extremely important.

MCP

Answers:

What can Claude do with Snyk?

Examples:

snyk_code_scan
snyk_sca_scan
snyk_iac_scan

Claude Code hooks

Answer:

When should security checks happen, and what should happen based on their results?

For example:

File changed



Snyk scan



New vulnerability?



YES



Tell Claude to fix it



Claude fixes



Scan again

Therefore:

MCP = capability layer

Hooks = enforcement/automation layer

Snyk's current Claude plugin uses both.

29. Current Snyk Secure-at-Inception Workflow

According to Snyk's current public Claude Code plugin, the Secure-at-Inception execution follows this structured sequence:

#	Stage / Action	Details & Logic Flow
1	File Modification	Claude edits a file in the user's workspace.
2	Background Scanning	A background Snyk scan is automatically triggered.

#	Stage / Action	Details & Logic Flow
3	Response Completion	Claude finishes its response generation.
4	Vulnerability Evaluation	The plugin checks for newly introduced vulnerabilities: • None found: Claude is allowed to finish normally. • Found: Claude is prevented from stopping, findings are provided to Claude, Claude fixes them, and a rescan is executed.

The current implementation caps the remediation cycle at three rounds to avoid an infinite loop. Dependency manifest changes can also trigger SCA scanning.

30. /snyk-fix

Snyk's Claude plugin also provides:

/snyk-fix

This is an on-demand remediation workflow.

According to the current plugin documentation, it can:

Scan for SAST/SCA vulnerabilities

↓

Select the highest-priority issue

↓

Fix the issue

↓

Run a validation scan

↓

Optionally open a PR

This demonstrates that Snyk's Claude integration isn't limited to:

Claude → scan → show result

It can participate in an entire:

scan → **analyze** → **fix** → **rescan**

development workflow.

31. Secure Dependency Health Check

The current Snyk Claude plugin also contains a:

Secure Dependency Health Check

This evaluates open-source packages when Claude is about to add a dependency.

It considers factors such as:

- 1. Vulnerability history**
- 2. Maintenance health**
- 3. Popularity**

and uses Snyk's package intelligence capabilities.

This shows that Snyk's overall Claude integration is becoming a broader **security guardrail system**, rather than merely an MCP scanner.

32. MCP Tool Profiles

Snyk also has different tool profiles.

The current source shows tools associated with profiles such as:

- 1. Lite**
- 2. Full**
- 3. experimental**

For example:

snyk_code_scan

- **full**
- **lite**
- **experimental**

snyk_sca_scan

- **full**
- **lite**
- **experimental**

snyk_container_scan

- **full**
- **experimental**

The exact availability depends on the selected profile and current version.

This provides a way to avoid unnecessarily exposing every possible security capability to an AI agent.

33. Why Tool Profiles Matter

Imagine exposing 100 tools to Claude:

tool1
tool2
tool3
...
tool100

That increases the amount of tool information the agent has to consider.

Instead, Snyk can provide a smaller set:

Lite

↓

Essential security tools

or:

Full



Broader security capabilities

This is a useful design consideration for any security product integrating with an AI agent.

34. Snyk Tool Descriptions Are Important

Snyk doesn't just give Claude a tool name.

The tool definitions contain detailed instructions.

For example, `snyk_sca_scan` tells the model:

- the path must be absolute;
- what types of project files it can inspect;
- when the tool should be used;
- that the package manager may need to be installed;
- which optional parameters are available.

Similarly, `snyk_code_scan` specifies:

- absolute path;
- supported severity thresholds;
- organization;
- ignored vulnerabilities;
- debugging options.

These descriptions are part of the MCP tool definitions.

This is important because **good MCP tool design is partly about good tool descriptions.**

35. Why Tool Descriptions Matter to an LLM

Consider:

snyk_code_scan

versus:

snyk_code_scan

Performs SAST on source code.

**Use during local development or after
generating new code.**

Required:

absolute path

Optional:

organization

severity threshold

include ignored issues

The second version gives Claude much more information to decide:

Should I use this tool?

When should I use it?

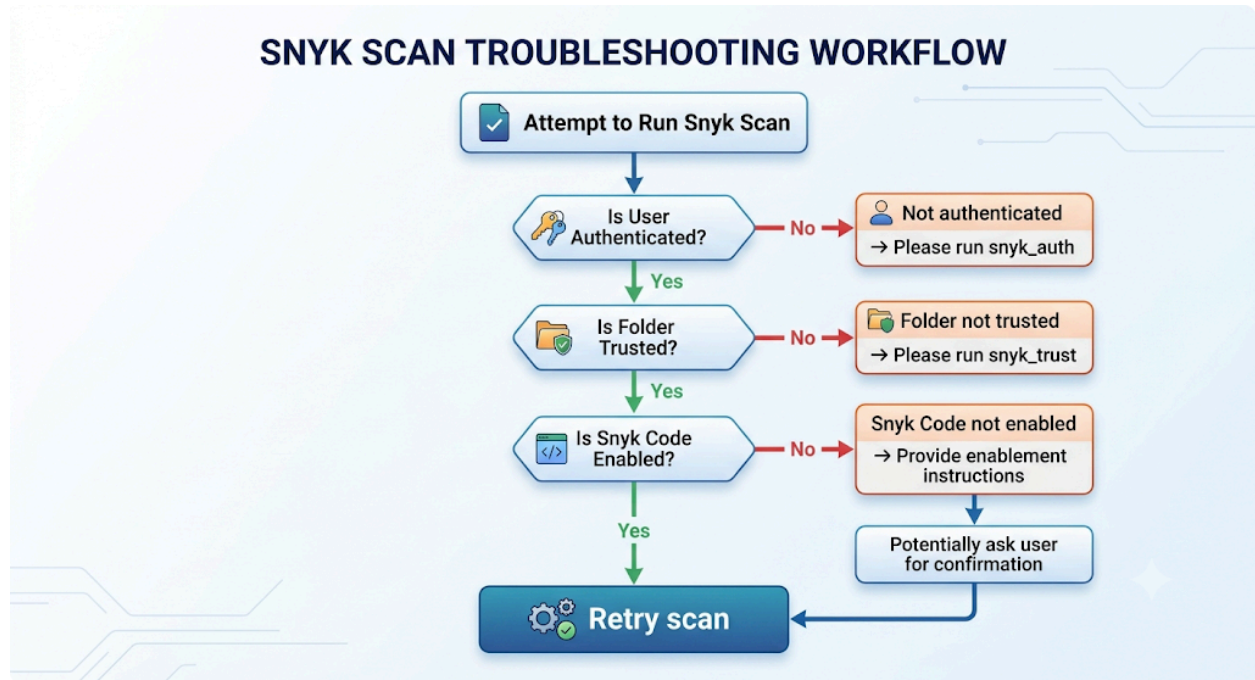
What parameters do I need?

How do I call it?

Therefore, the MCP server's tool schema is part of the AI integration design.

36. Error Handling

The MCP server also handles situations such as:



The current source contains logic specifically for Snyk Code enablement when a scan encounters the corresponding condition.

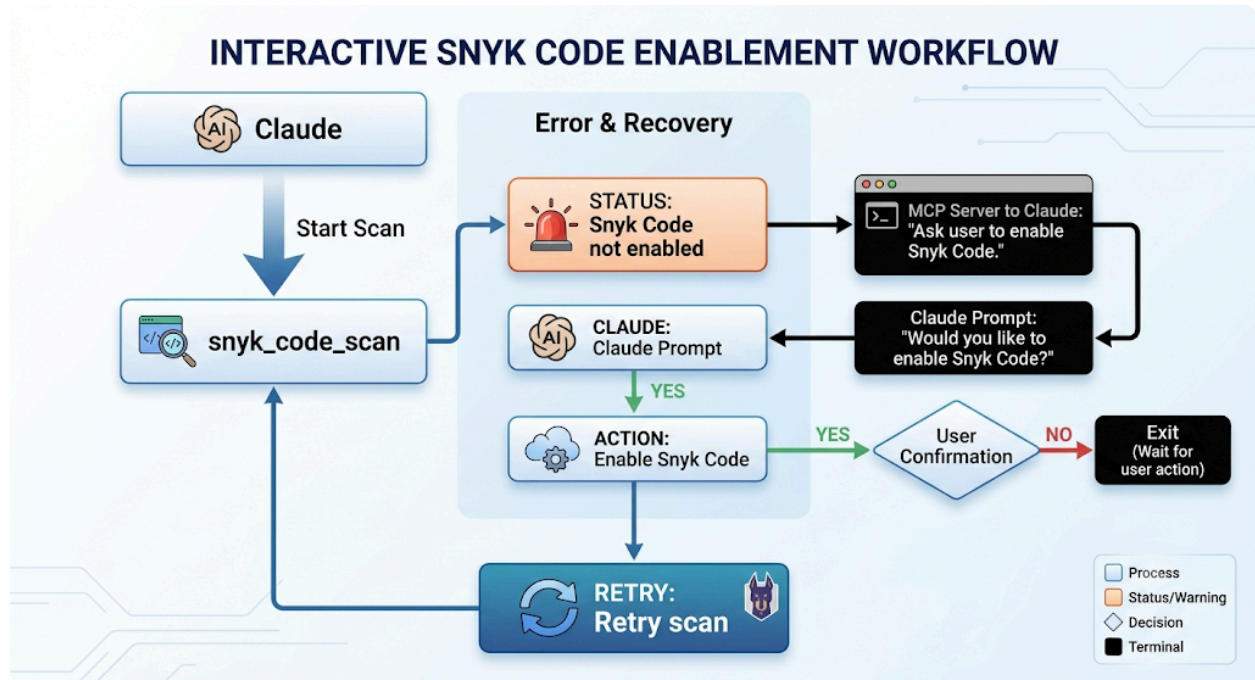
This is another important design principle:

The MCP server should provide meaningful, AI-consumable errors rather than simply exposing raw command failures.

37. Snyk Code Auto-Enablement

The current implementation has special handling for a Snyk Code-not-enabled condition.

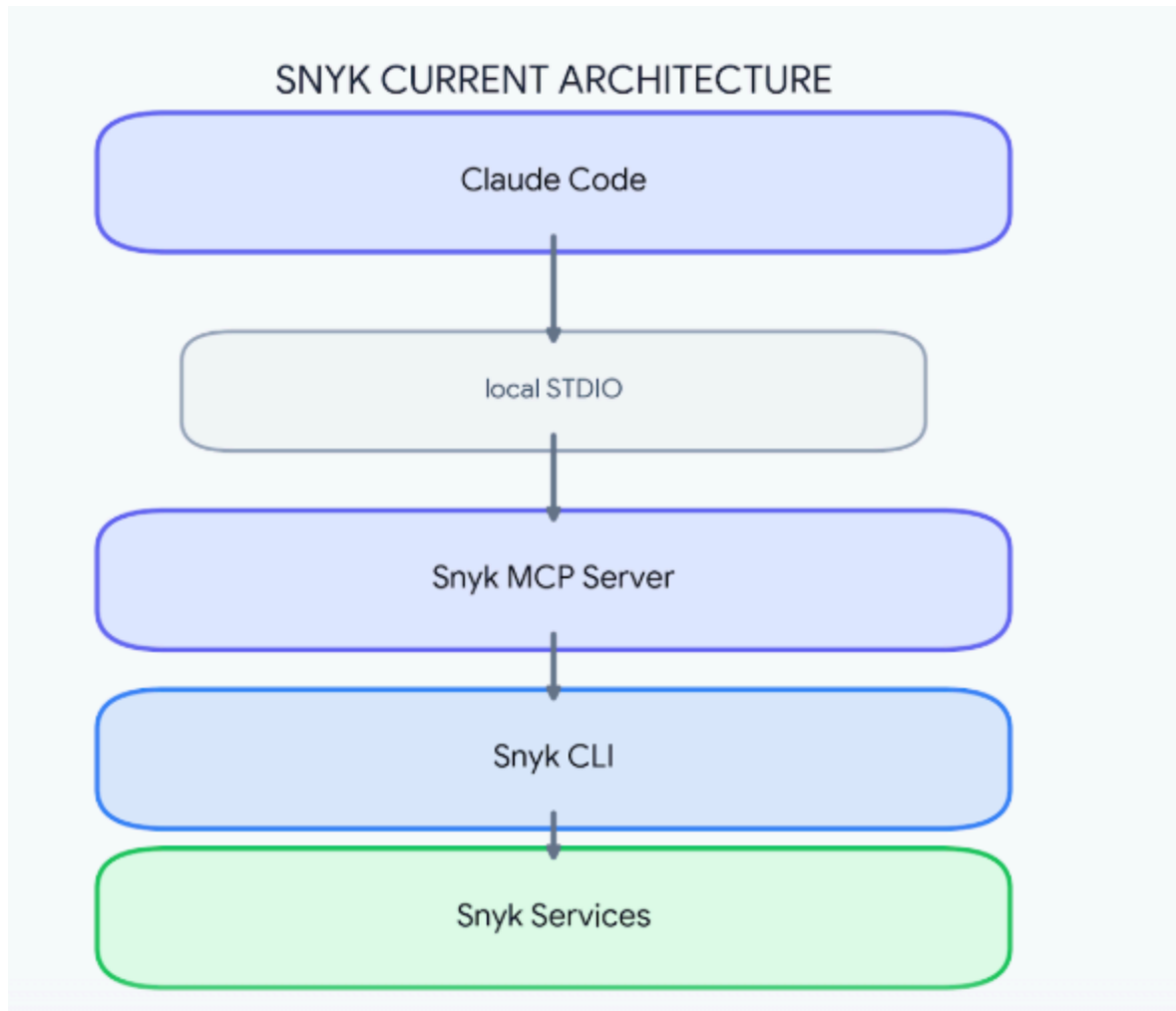
Conceptually:



Snyk's source explicitly implements this workflow.

38. Local vs Remote MCP Architecture

Snyk's current architecture is:



It is **not** currently a hosted **Snyk MCP server** such as:

Claude



Snyk Cloud MCP Server

Snyk's documentation explicitly states that it does not offer a hosted/remote version of its MCP server and that the server is designed to run locally through the Snyk CLI for local file access.

39. Why Local MCP Makes Sense for Snyk

Consider a developer working on:

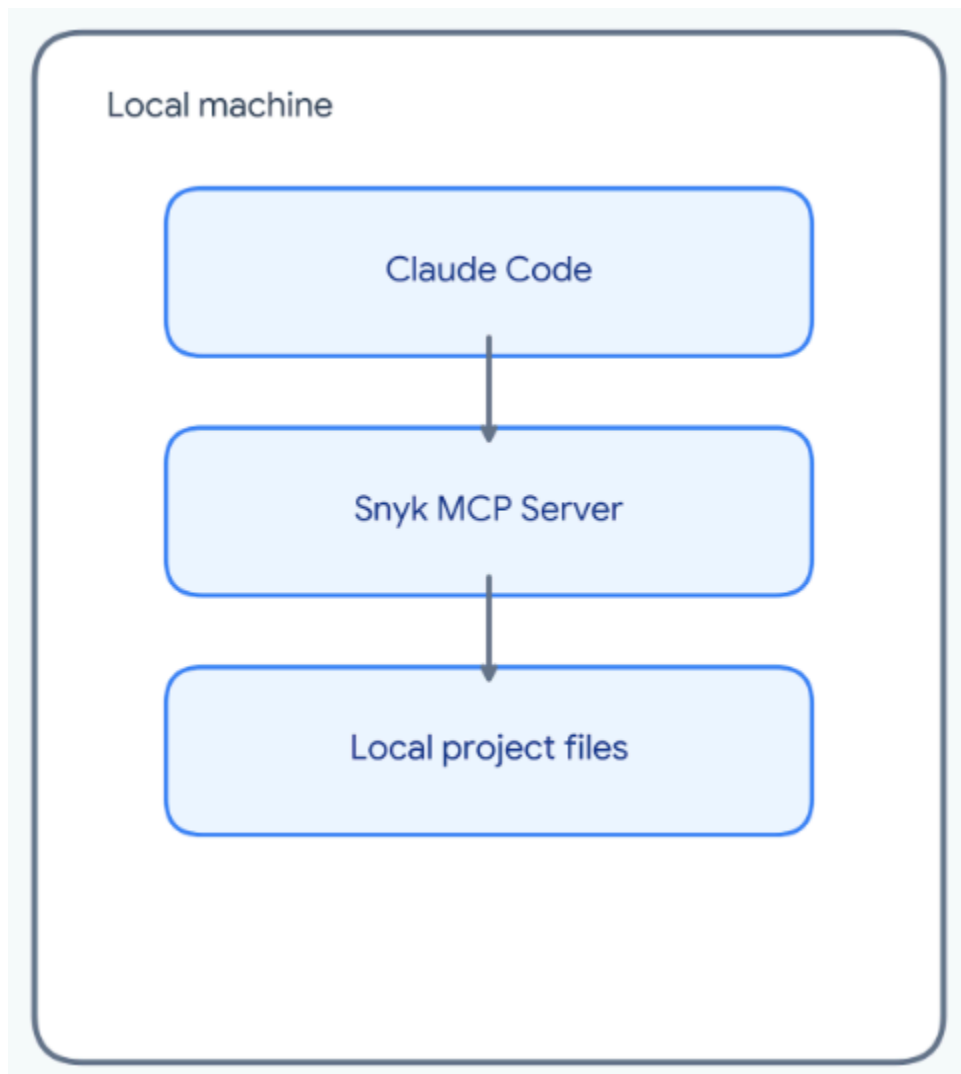
/Users/developer/projects/myapp

The code may not yet be pushed anywhere.

Claude can access the local project.

Snyk can therefore scan it locally.

Architecture:



This is one reason Snyk chose the local MCP architecture.

40. Authentication Environment

Snyk's MCP bundle also allows an optional:

SNYK_TOKEN

configuration.

The current bundle manifest describes this as an optional fallback, while recommending interactive authentication through `snyk_auth`.

Conceptually:

Option 1

Claude

↓

snyk_auth

↓

Snyk authentication

or:

Option 2

SNYK_TOKEN

↓

Snyk MCP Server

↓

Snyk authentication

41. Full Example: Developer Adds a Vulnerable Dependency

Suppose the developer asks Claude:

"Add the latest version of package X."

Claude modifies:

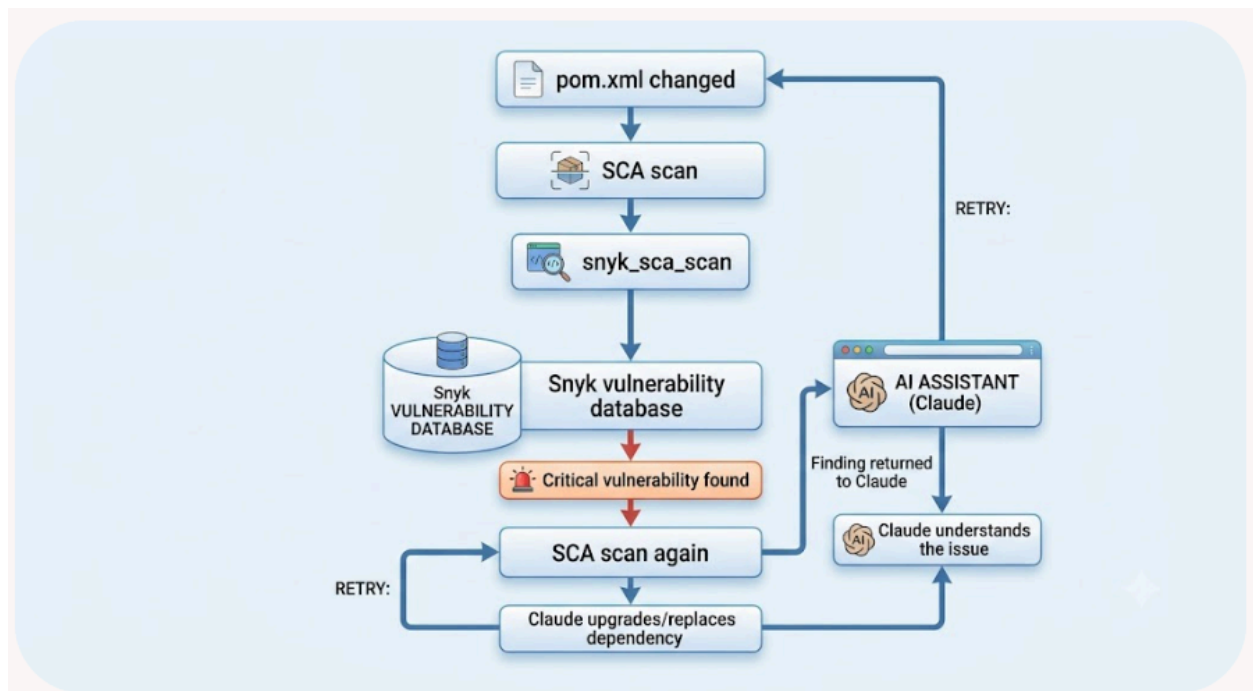
pom.xml

and adds:

```
<dependency>
  <groupId>example</groupId>
  <artifactId>package-x</artifactId>
  <version>1.2.0</version>
</dependency>
```

The Snyk Secure-at-Inception workflow can detect a dependency manifest change.

Then:



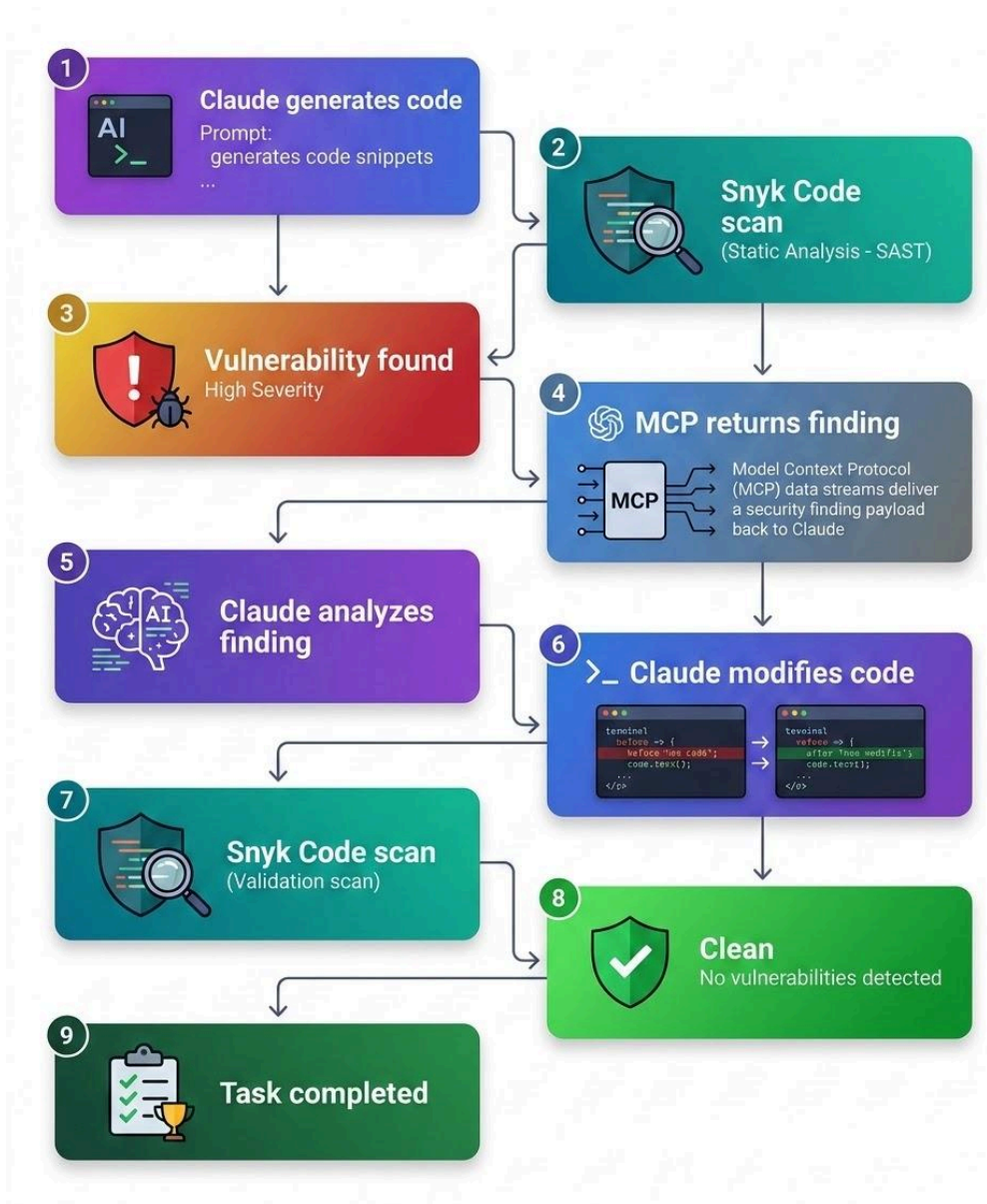
The current Snyk Claude plugin explicitly documents dependency manifest changes triggering SCA scans.

42. Full Example: Developer Creates Vulnerable Code

Developer:

"Create a login endpoint."

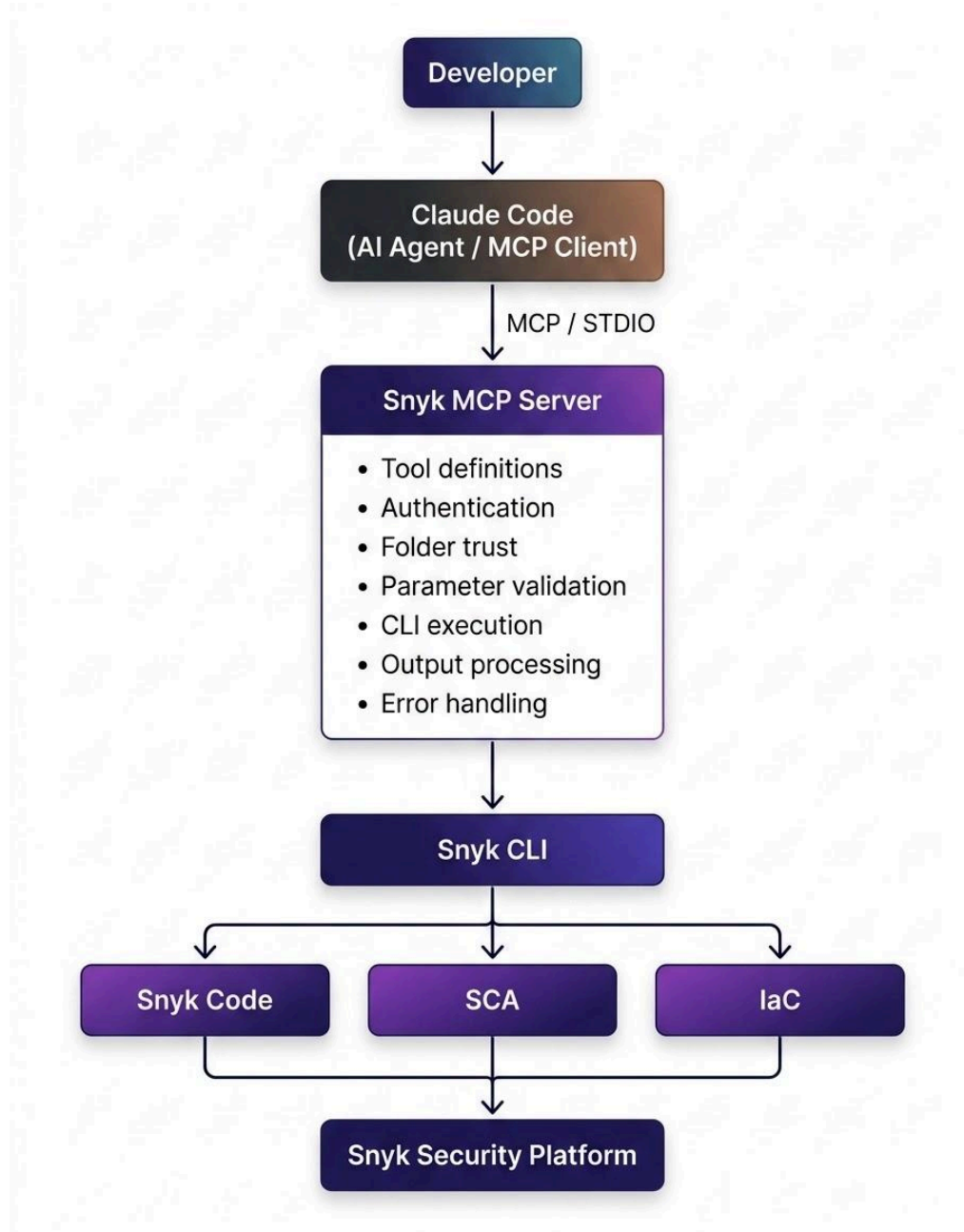
Claude generates code.



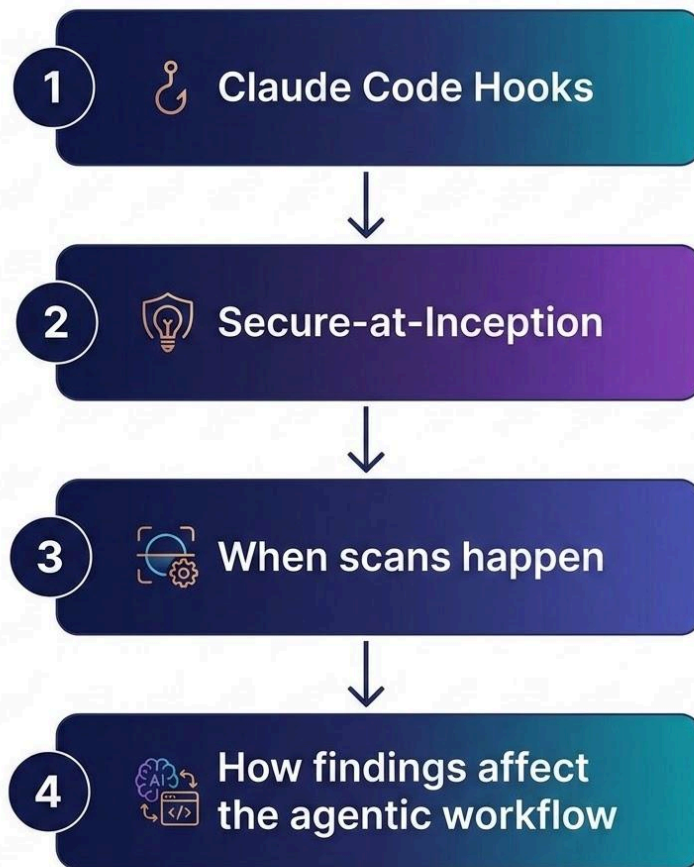
This is the fundamental **agentic security loop**.

43. The Complete Architecture

Putting everything together:

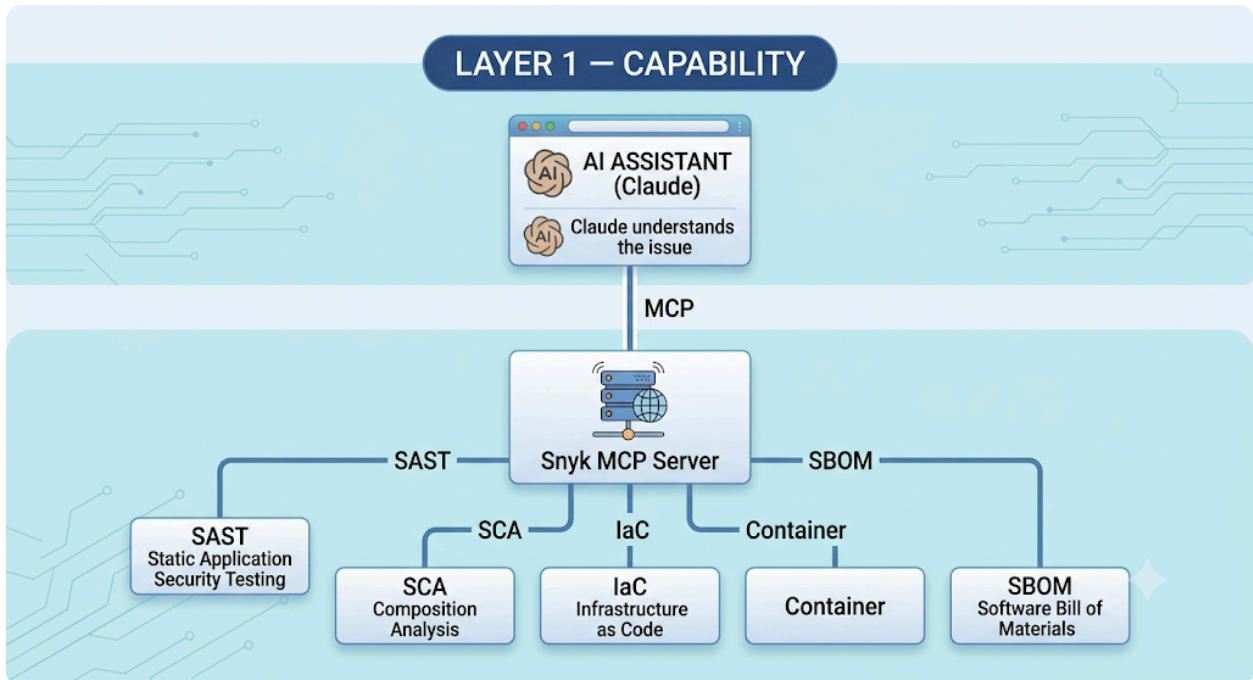


And alongside this:



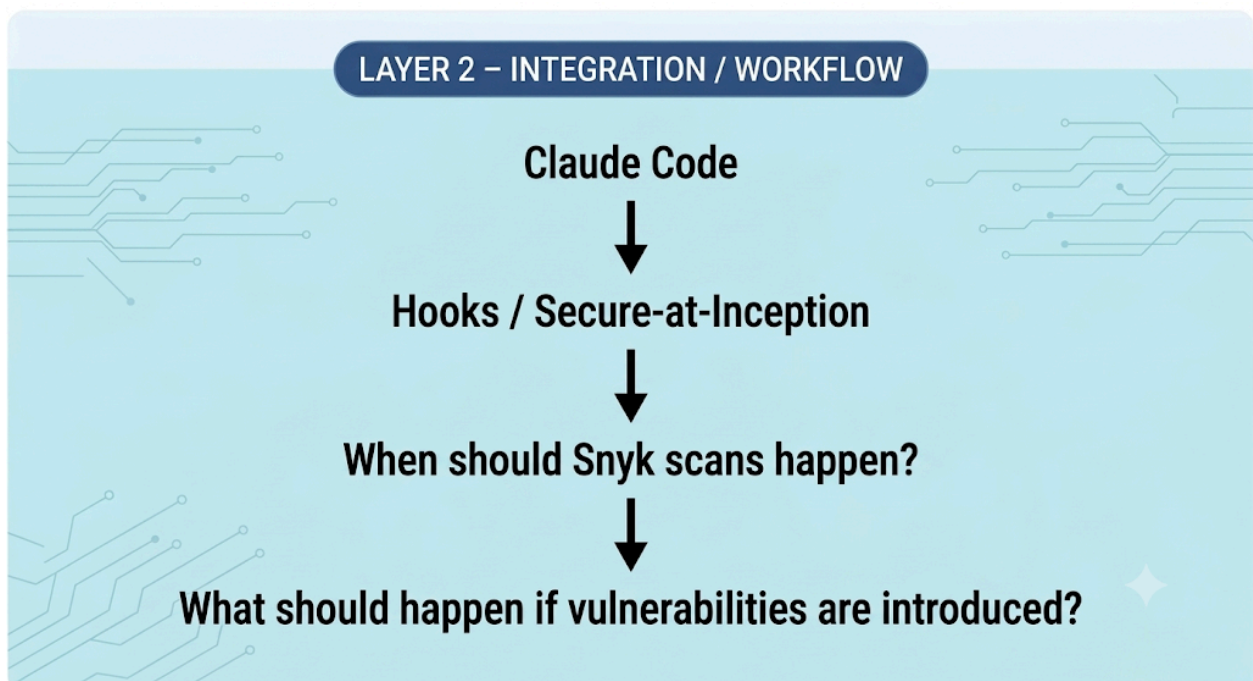
44. The Two-Layer Snyc + Claude Integration

The cleanest way to understand the integration is as **two layers**.



This answers:

What can Claude do with Snyp?



This answers:

When and how should those capabilities be used during development?

Snyk's current Claude plugin explicitly combines these capabilities.

45. What Snyk Has Actually Built

It would be inaccurate to describe the work simply as:

"Snyk integrated with Claude."

A more technically accurate description is:

Snyk provides a local MCP server through its CLI. Claude Code acts as the MCP client and launches the Snyk MCP server over STDIO. The MCP server exposes Snyk security capabilities as typed tools such as `snyk_code_scan` and `snyk_sca_scan`. Those tools invoke Snyk's underlying CLI/scanning capabilities and process the results before returning them to Claude. Snyk additionally provides Claude Code hooks and other plugin components to integrate those security capabilities into the agentic development workflow through its Secure-at-Inception functionality.

This is supported by Snyk's current documentation and public implementation.

46. What MCP Is NOT

MCP is **not**:

A scanner

A vulnerability database

An AI model

A Claude plugin by itself

A security engine

MCP is:

A standardized interface through which an AI client can discover and invoke external capabilities.

In this case:

Claude + MCP + Snyk

creates the integration.

47. Key Architectural Lessons from Snyk

There are several important lessons from this implementation.

47.1 Don't build a Claude-specific security engine

Instead:

Existing security engine + MCP server

can make the existing capability available to Claude.

47.2 Expose specialized tools

Prefer:

code_scan

sca_scan

iac_scan

container_scan

over one huge:

generic_scan

tool.

47.3 Provide detailed tool descriptions

Tell the AI:

What the tool does

When to use it
Required parameters
Optional parameters
Parameter constraints
Expected behavior

47.4 Make the output AI-friendly

Don't simply dump:

50,000 lines of CLI output

into the model.

Use an MCP adapter to provide useful security findings.

47.5 Handle authentication inside the integration

Provide an explicit capability such as:

auth

rather than expecting users to manually understand the underlying authentication mechanism.

47.6 Protect local resources

Snyk's folder-trust mechanism demonstrates that an AI-accessible security tool should consider which local paths the agent is allowed to operate against.

47.7 Separate capability from workflow

MCP provides:

"What can I do?"

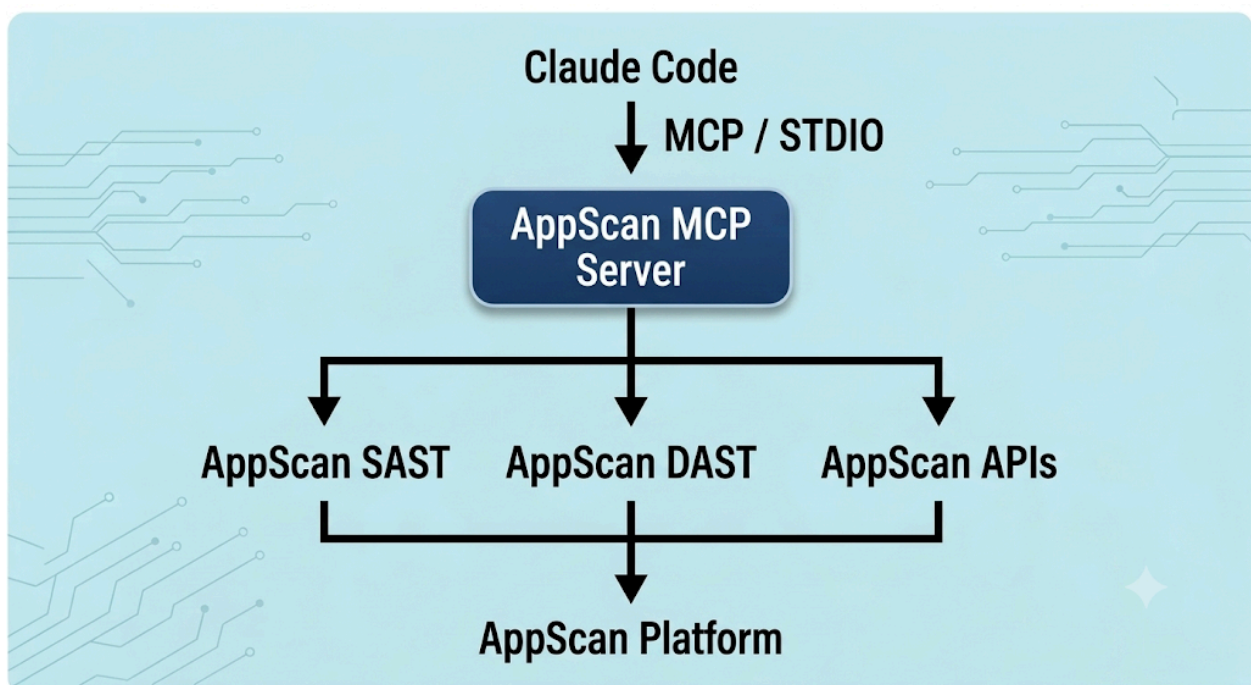
Hooks/rules/workflows provide:

"When should I do it?"

This separation is extremely useful.

48. Implications for an AppScan Integration

If the purpose of studying Snyk is to understand how a product such as AppScan could integrate with Claude, the Snyk architecture suggests something like:

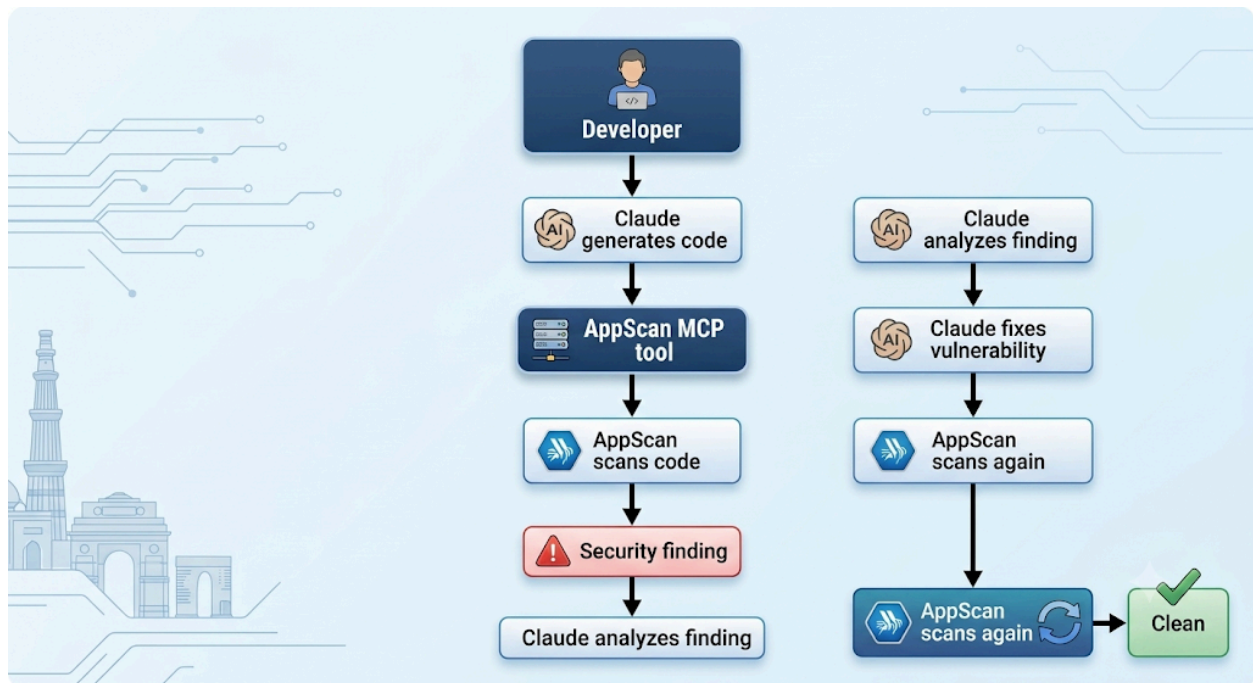


Potential tools could conceptually be:

`appscan_code_scan`
`appscan_dast_scan`
`appscan_get_findings`
`appscan_get_finding`
`appscan_auth`
`appscan_project_info`

The exact tools would need to be determined based on the AppScan APIs/CLI and the desired developer workflow.

49. Potential AppScan Agentic Workflow



The security product remains responsible for:

Security analysis

while Claude remains responsible for:

Reasoning

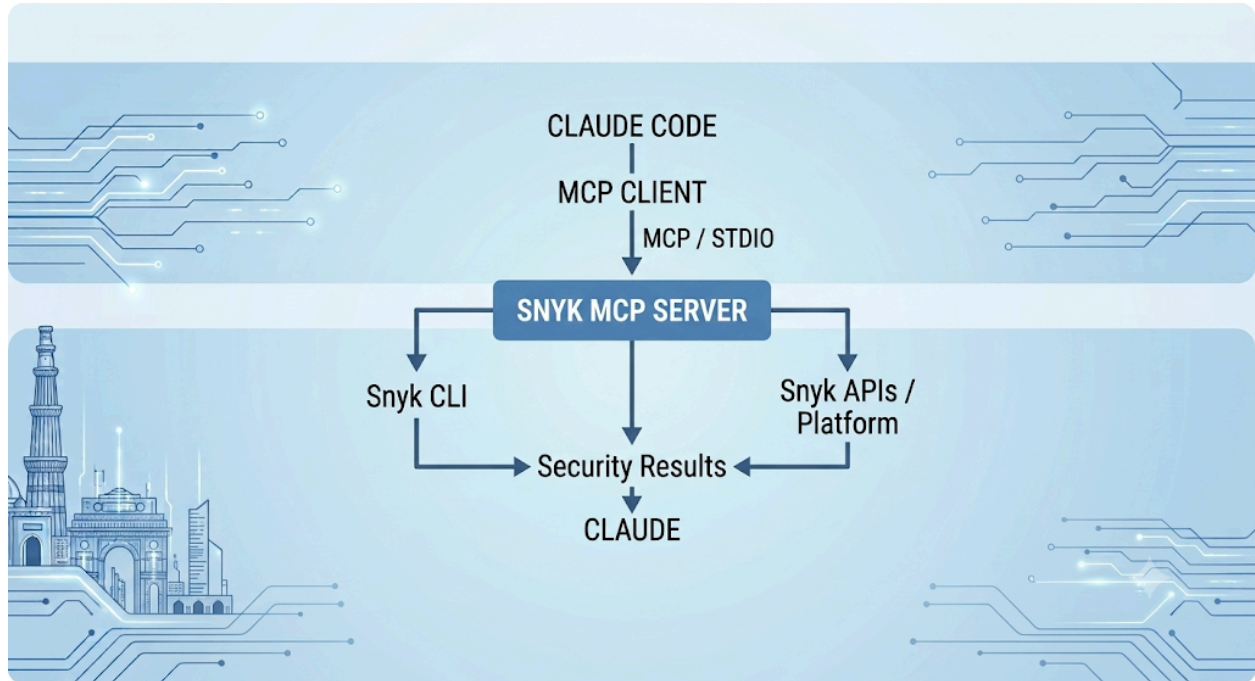
Coding

Remediation

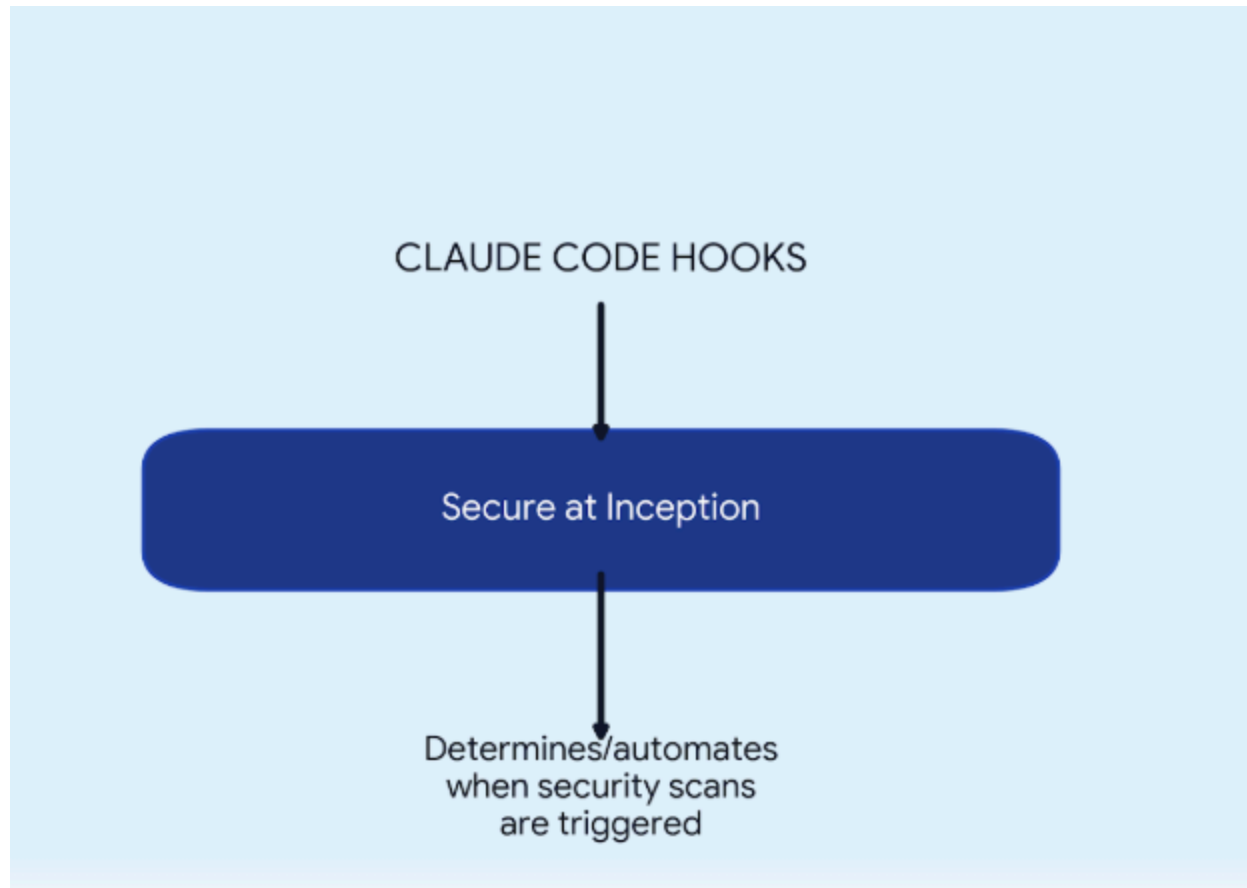
and MCP provides the connection between them.

50. Final Mental Model

The simplest way to remember the entire Snyk + Claude architecture is:



And separately:



So the fundamental architecture is:

Claude Code = AI/MCP client

Snyk MCP Server = security capability adapter

Snyk CLI/Platform = actual security engine

MCP = communication protocol

Claude Code hooks = agentic workflow/enforcement mechanism

That is the most accurate way to describe **how Snyk currently integrates with Claude Code** based on its public documentation and source as of September 2026.

Primary References

Snyk – Claude Code Guide

<https://docs.snyk.io/integrations/developer-guardrails-for-agentic-workflows/quickstart-guides-for-mcp/claude-code-guide>

Snyk – Getting Started with Snyk Studio

<https://docs.snyk.io/integrations/snyk-studio-agentic-integrations/getting-started-with-snyk-studio>

Snyk Studio MCP – GitHub Repository

<https://github.com/snyk/studio-mcp>

Snyk Claude Code Plugin – GitHub Repository

<https://github.com/snyk/claude-plugin-snyk>

Snyk MCP Tool Definitions

https://github.com/snyk/studio-mcp/blob/main/internal/mcp/snyk_tools.json

Snyk Claude Code MCP Configuration

<https://github.com/snyk/claude-plugin-snyk/blob/main/.mcp.json>

Claude Code – MCP Documentation

<https://code.claude.com/docs/en/mcp>

Snyk User Docs – Agentic/MCP Integrations

<https://docs.snyk.io/integrations/snyk-studio-agentic-integrations>

The current public Snyk Claude Code plugin uses **MCP plus Claude Code hooks, commands and skills**, with hooks being a significant part of the current Secure-at-Inception implementation.